UMDE Rule Extraction Engine: Locked Strategy (Baseline)

Objective

To extract and normalize CMS validation rules from heterogeneous document types (PDF, XLSX), and transform them into accurate, deterministic, and executable rule files used by the Unified Member Data Engine (UMDE).

Source Documents

- PDFs: PCUG (Plan Communication User Guide), EDPS Guide, 837 Companion Guides
- Excel: CEM 837 Institutional, Professional, DME Edit Spreadsheets
- Future-ready: Any designated CMS source document

Input Handling

Model Complexity Note: *To mitigate maintenance risk from a multi-model pipeline (Zephyr → TinyLama → FlanT5), each model is version-controlled. Metadata includes model version, fallback source, and extraction confidence. Retraining or replacement decisions will be based on fallback volume and extraction quality thresholds.*

- Multi-format ingestion engine (PDF + XLSX)
- Model-driven extraction pipeline: *Zephyr → TinyLama → FlanT5*→ Heuristic fallback
- Deterministic fallback path to rule-based parser and static templates if AI fails
- Batch inference: Extract multiple rule descriptions at once (In Progress)
- Parallel model execution via asyncio or thread pool (Planned)
- RAG result caching using Redis or hash-indexed disk cache (Planned)

Scope of Rule Extraction

- Only field-level validation rules are extracted
- Two primary categories:
    1. Member-specific TRC rules
    2. 10 Configurable Critical Fields (UI-configurable; default list includes):
        - DiagnosisCode
        - ProcedureCode
        - ServiceDate
        - NPI
        - HICN / MBI

- Member First Name
- Member Last Name
- Payer ID
- Place of Service
- Claim Amount

Output Structure

UMDE Integration Note: *YAML and JSON outputs follow schema contracts aligned with the UMDE validator engine. Integration tests validate: schema conformance, field resolution traceability, and full rule roundtrip: YAML → validator → backtrace.*

YAML Rule Tier Tagging (Expanded)

```yaml
rule_tiers:
  Level 1:
    description: Simple field checks (e.g., required, basic format)
    generation: auto
  Level 2:
    description: Moderate logic (cross-field consistency, regex validation)
    generation: auto
  Level 3:
    description: Conditional rules, field dependencies, multi-segment logic
    generation: manual_stub
  Level 4:
    description: Complex sequence logic, rule chaining, external context
resolution
    generation: manual_stub
```

cms_rules.yml

A YAML configuration file listing all enabled rules and tags:

```yaml
enabled_rules:
  - TRC004
  - TRC203
  - EDPS_2300_CLM02_Required

tags:
  TRC004:
    - member
    - cms-critical
  EDPS_2300_CLM02_Required:
    - edps
    - claim
    - required
```

minimal_rules.json

A clean, production-ready JSON format with traceability:

```
{
  "rule_id": "TRC004",
  "title": "Member Gender Validation",
  "short_definition": "Must be M, F, or U",
  "definition": "The member gender field must be one of M, F, or U.",
  "field": "member.gender",
  "plan_action": "reject",
  "layer": "1",
  "tags": ["member", "high-risk"],
  "severity": "error",
  "doc_link": "https://www.cms.gov/files/document/plan-communications-user-guide-v178.pdf#page=167",
  "meta": {
    "source_page": 167,
    "classification_source": "FLAN-T5",
    "layer_reason": "matches known gender logic",
    "raw_row": "TRC004 - Member gender must be M, F, or U.",
    "confidence": "0.91",
    "extraction_chain": ["Zephyr", "FLAN-T5"]
  }
}
```

Rule Code Generation

Performance & Load Testing Note: *While parallelism and batch inference are planned, performance benchmarking (inference latency, throughput, memory use) is scheduled post-M1 delivery. Target throughput: 500+ rules/page/min under load. Stress tests will simulate 5,000+ TRCs concurrently.*

Rule Level	Action
Level 1 & 2	Full code auto-generated
Level 3 & 4	Code stub generated for manual completion

- Generated code is fully testable and routed into the validator
- Stub files include TODO + metadata block
- All code generation must be 100% accurate and deterministic

---

Arbitration & Fallback Trace

- Confidence thresholds guide fallback decisions (e.g., Zephyr score < 0.85 triggers fallback)
- All arbitration steps logged with model name, score, and fallback reason
- UI override possible (force mapping, manual correction) (Implemented)
- Logs store which model was ultimately selected (Will feed fine-tuning loop)

Rule Testing & Validation

Error Monitoring Note: *Fallback frequency is logged per rule type and source document. Any TRC or EDPS rule with a 25%+ fallback rate triggers a fine-tuning candidate flag. Future versions will support automated dataset curation based on low-confidence segments.*

- Rules are tested using `RuleAccuracyTester`
  - Executes test cases per rule
  - Tracks false positives/negatives
  - Reports rule-level accuracy and execution time
- CI/CD hooks enforce coverage per tier

Auditability & Logs

Security & Privacy Note: *UMDE handles CMS documents and potentially identifiable information. All rule outputs are encrypted at rest (S3, Redis) and in transit (HTTPS, TLS 1.2+). Role-based IAM policies are enforced across all microservices. The system design aligns with HIPAA and HITRUST compliance baselines.*

- All extracted rules include:
  - TRC/EDPS source + page reference
  - Timestamp, confidence, extraction duration
  - Execution logs per rule in `cms_parser.log`
- Audit files: `rule_metadata.json`, `audit_events.parquet`, `validation_trace.parquet`

UI Explainability Features (Planned)

User Support Note: *UMDE will provide guided tooltips, onboarding walkthroughs, and contextual help integrated in the UI. Additionally, video walkthroughs and a "What is this rule?" explainer will support adoption by non-technical users.*

- Side-by-side model output comparison
- Highlighted diffs across fallback outputs
- "Why this rule?" button → opens CMS snippet or RAG source
- Inline confidence score badges for transparency

837-Type Specific Critical Field Support

```yaml
critical_fields:
  837I:
    - DiagnosisCode
    - AdmissionDate
```

```
    - BillingProviderNPI
    - TypeOfBill
    - ClaimAmount
  837P:
    - DiagnosisCode
    - ProcedureCode
    - ServiceDate
    - RenderingProviderNPI
    - PlaceOfService
    - ClaimAmount
  837DME:
    - ProductCode
    - SupplierNPI
    - CertificateReference
    - ServiceDate
    - DiagnosisCode
    - ClaimAmount
```

Strategic Impact (EB-2 NIW Readiness)

- Targeted reduction in rule extraction errors: 30% fewer invalid validations
- Estimated financial impact: ≥ $10M/year in savings
- Supports CMS audit traceability
- Quote and penalty data sourcing (Underway)

Summary

Future Innovation: Adaptive Intelligence & Governance

Real-Time Learning / Self-Tuning Rules UMDE will monitor fallback frequency per rule and dynamically adjust extraction routing. For example, if `TRC257` falls back >25% of the time, the system can re-prioritize models or trigger fine-tuning. This enables routing that evolves over time without altering CMS content.

Format Conflict Detection (PDF vs XLSX) In cases where rules are defined across formats (e.g., 837 Professional Excel vs. PCUG), UMDE will use semantic comparison to flag discrepancies. These are logged in `validation_trace.parquet` and surfaced in the UI, allowing humans to resolve contradictions while maintaining compliance.

Real-Time Update Awareness UMDE will track official CMS sources (e.g., CMS.gov document updates). When a new version is detected (e.g., PCUG v17.9), the system will:

- Download and version-store the file (HIPAA-compliant)
- Compare rules using AI-assisted text diff
- Re-extract impacted rules

- Notify users of affected TRCs (with UI explanation links)

This ensures automatic alignment with CMS guidance, traceable diffs, and reviewer visibility—all without compromising source authority.

This strategy outlines a Phase-1 complete roadmap for UMDE's CMS rule extraction engine—fully integrating multi-model intelligence, fallback orchestration, rule generation, traceability, and reviewer-ready instrumentation.

This isn't a prototype. This is launch-stage infrastructure.